

PROYECTO FINAL DE PROGRAMACIÓN

Sistema de Invernadero Inteligente y Automatizado

Jonathan Ramirez Velez, Esteban henao, Oscar David Obando, Juan Sebastian Ladino,

Visión General del Proyecto

Optimización Agrícola

Desarrollo de un ecosistema digital capaz de gestionar variables críticas para la producción de Tomate, Lechuga y Pimiento.

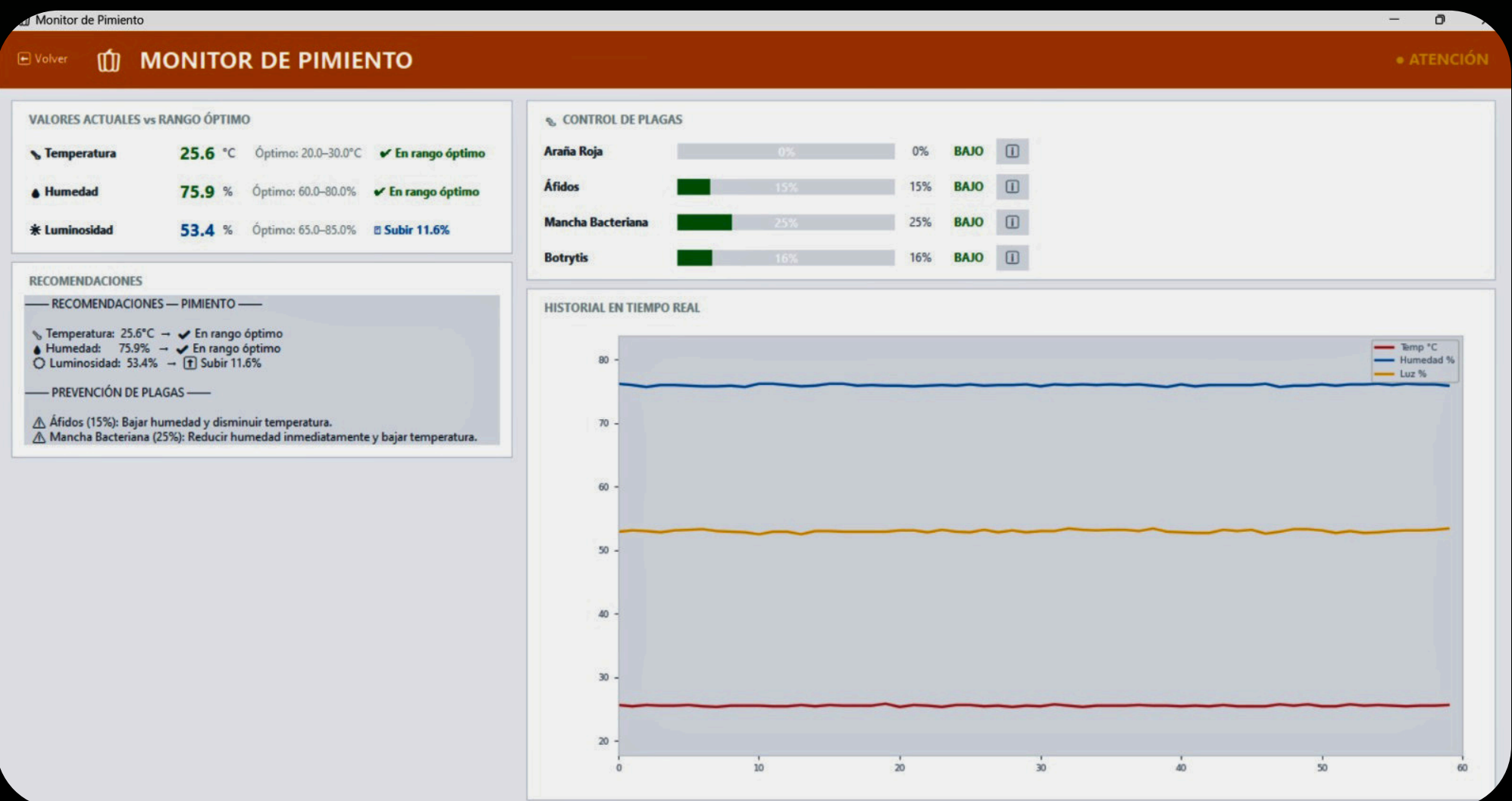
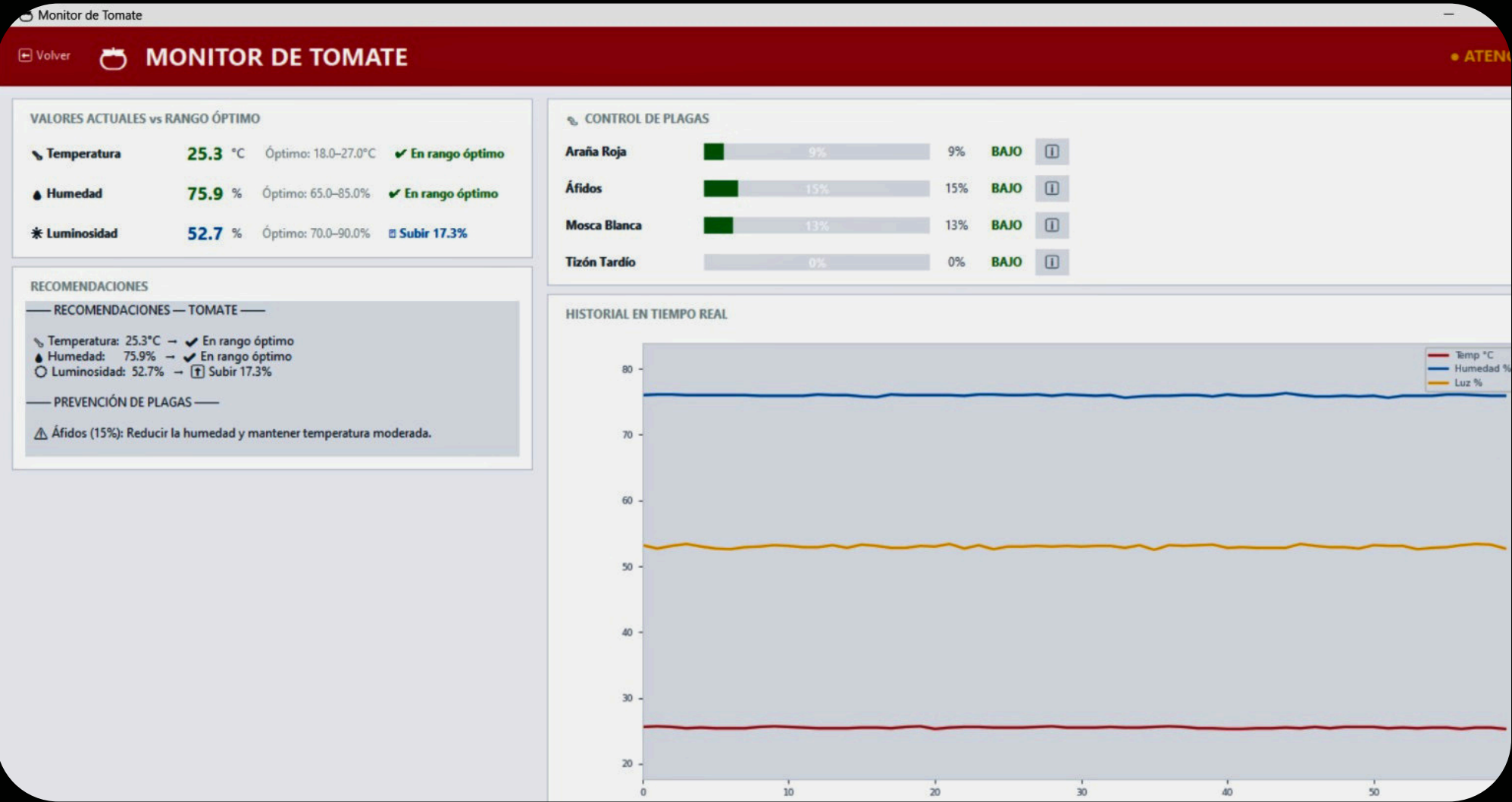
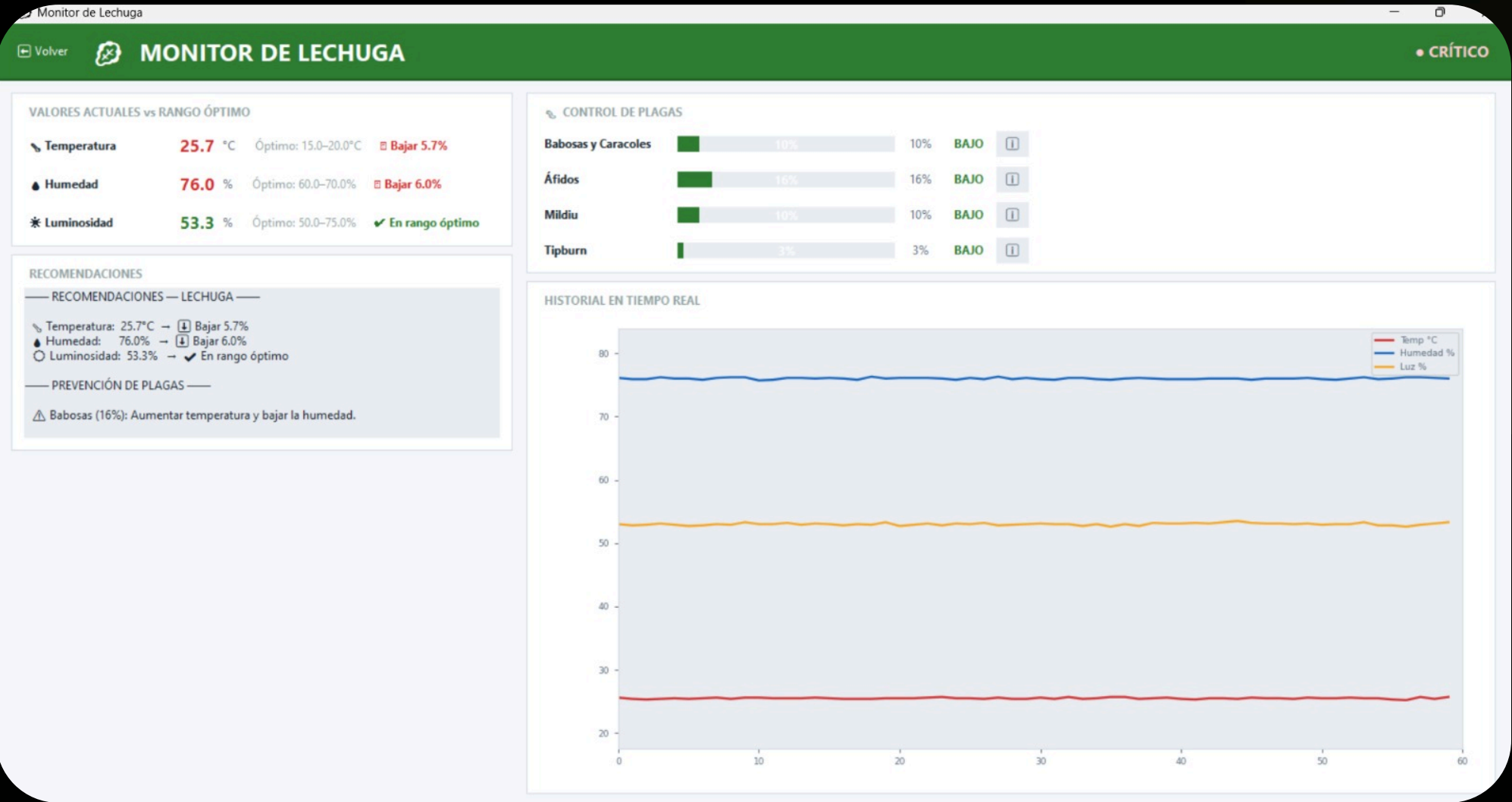
- Control autónomo de microclima.
- Prevención algorítmica de plagas.
- Interfaz gráfica de alta fidelidad.
- Backend robusto de procesamiento de datos.



Gestión de Cultivos Específicos

El sistema adapta sus algoritmos según el cultivo seleccionado, modificando las Recomendaciones y el Control de Plagas:

Cultivo	Temp. Óptima	Hum. Óptima
Tomate	18°C - 27°C	60% - 80%
Lechuga	15°C - 20°C	50% - 70%
Pimiento	20°C - 25°C	65% - 85%



Arquitectura del Sistema

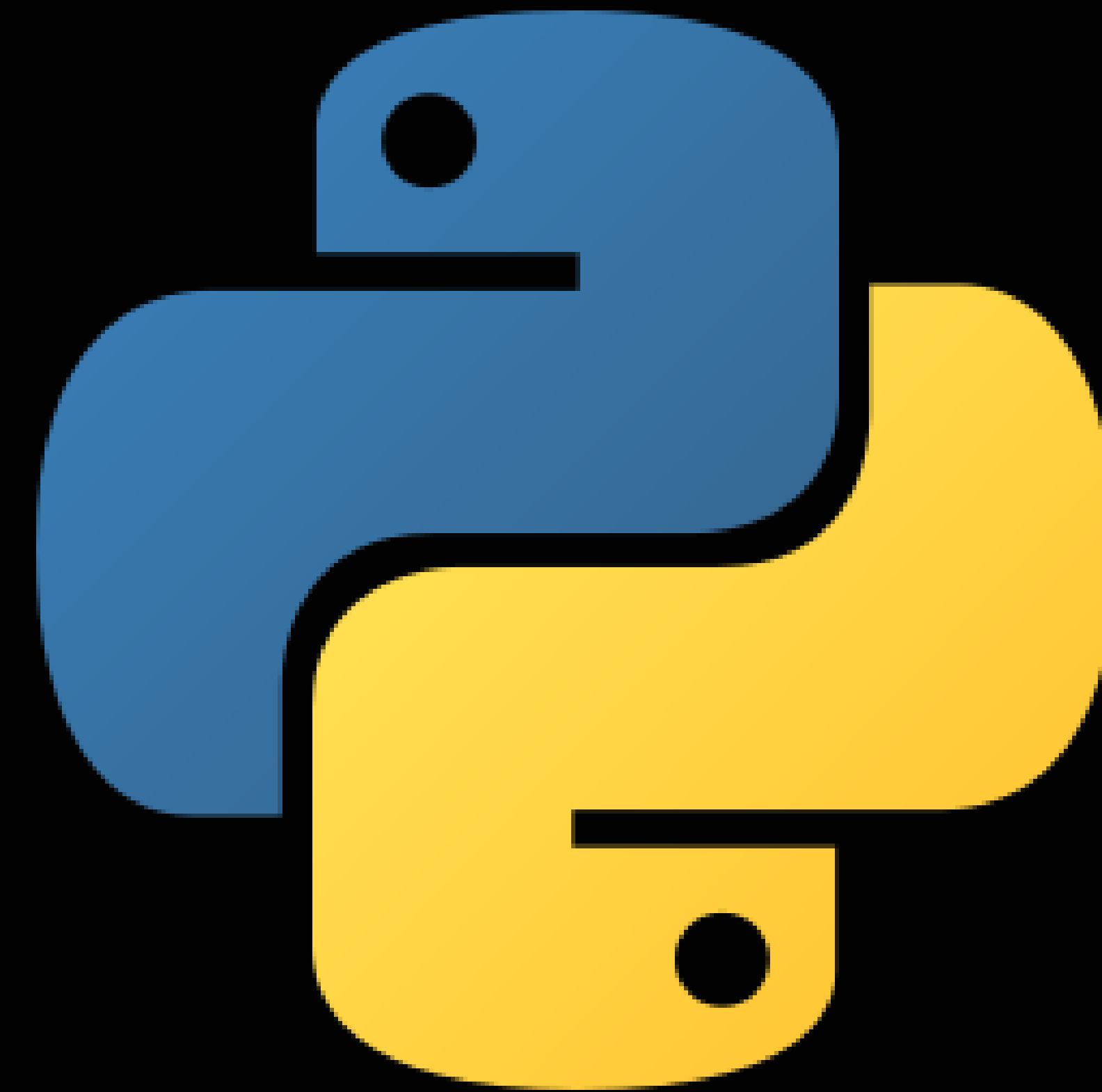
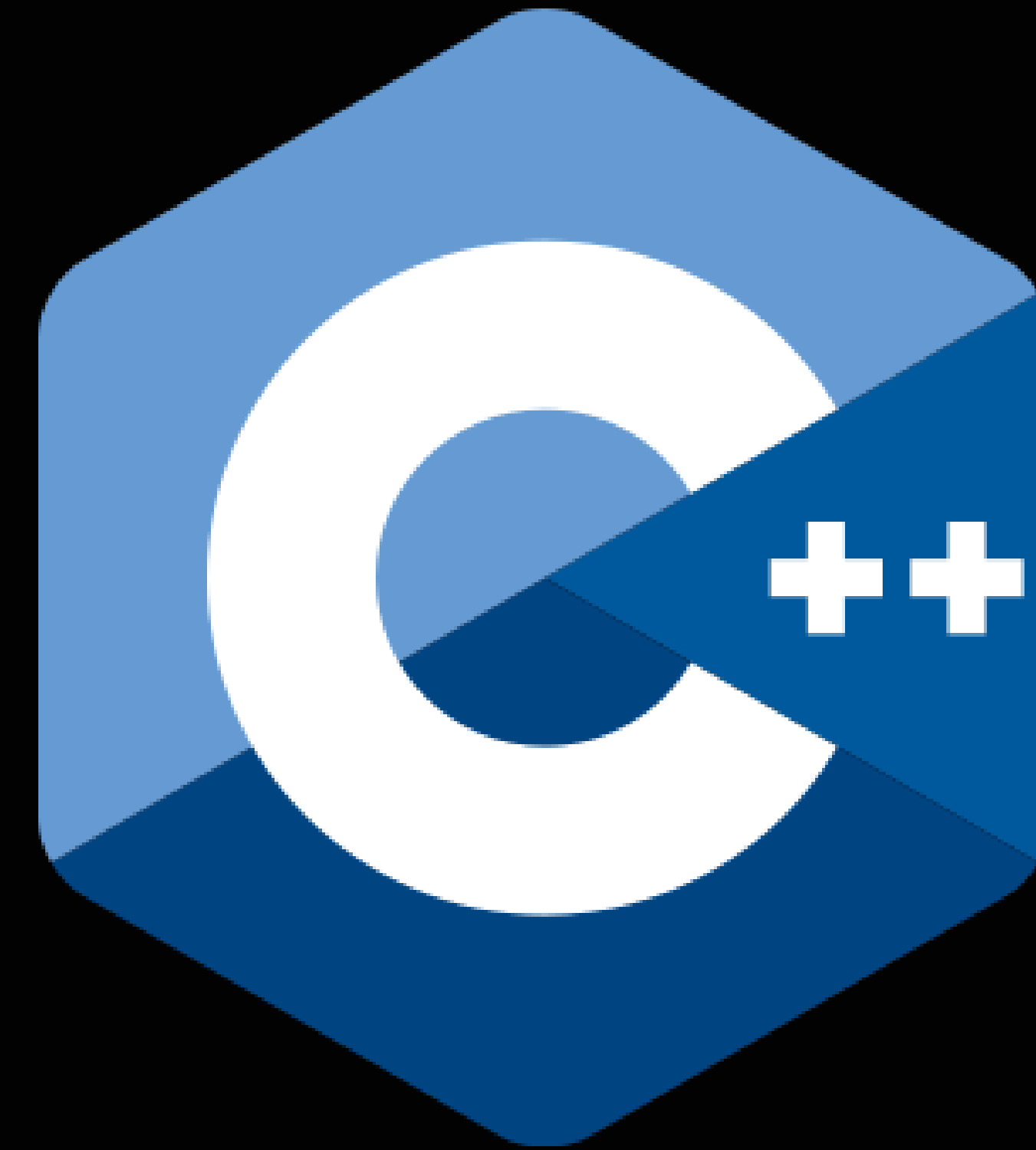
El proyecto emplea una estrategia de Programación Híbrida:

Python (Frontend/Control)

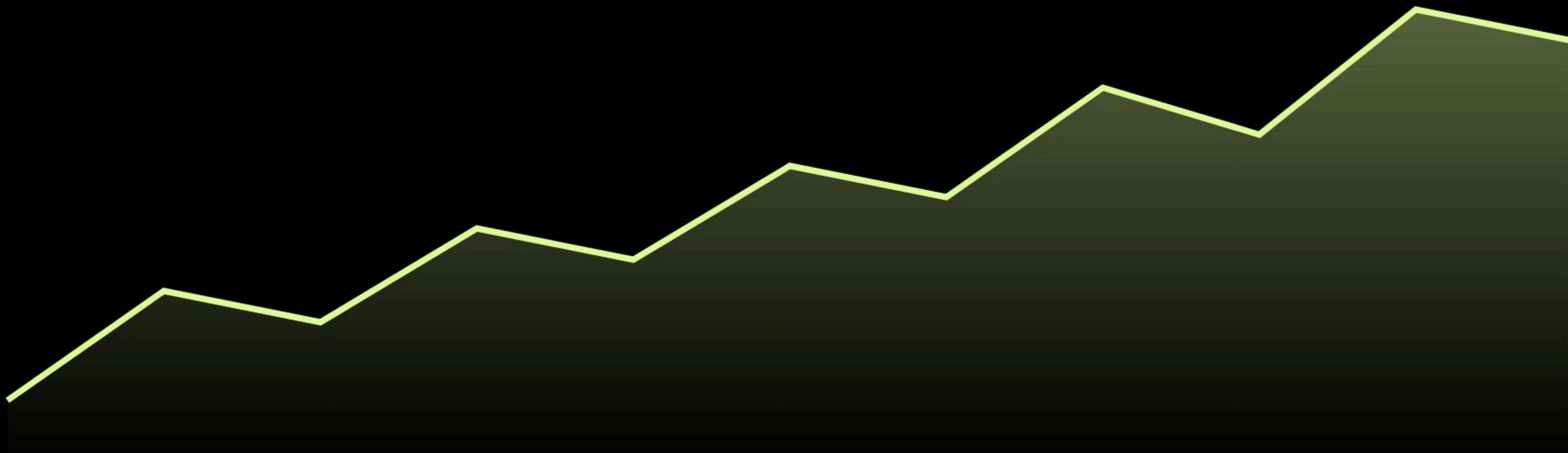
Gestión de GUI, hilos de ejecución y orquestación de sensores.

C++ (Backend)

Cálculo intensivo, generación de gráficas y lógica de control de plagas para máxima eficiencia.



| Telemetría y Tendencias en Tiempo Real



Las gráficas son generadas mediante el motor C++ (`graficas.cpp`) para asegurar una respuesta fluida en la interfaz Python.

| Lógica de Prevención de Plagas

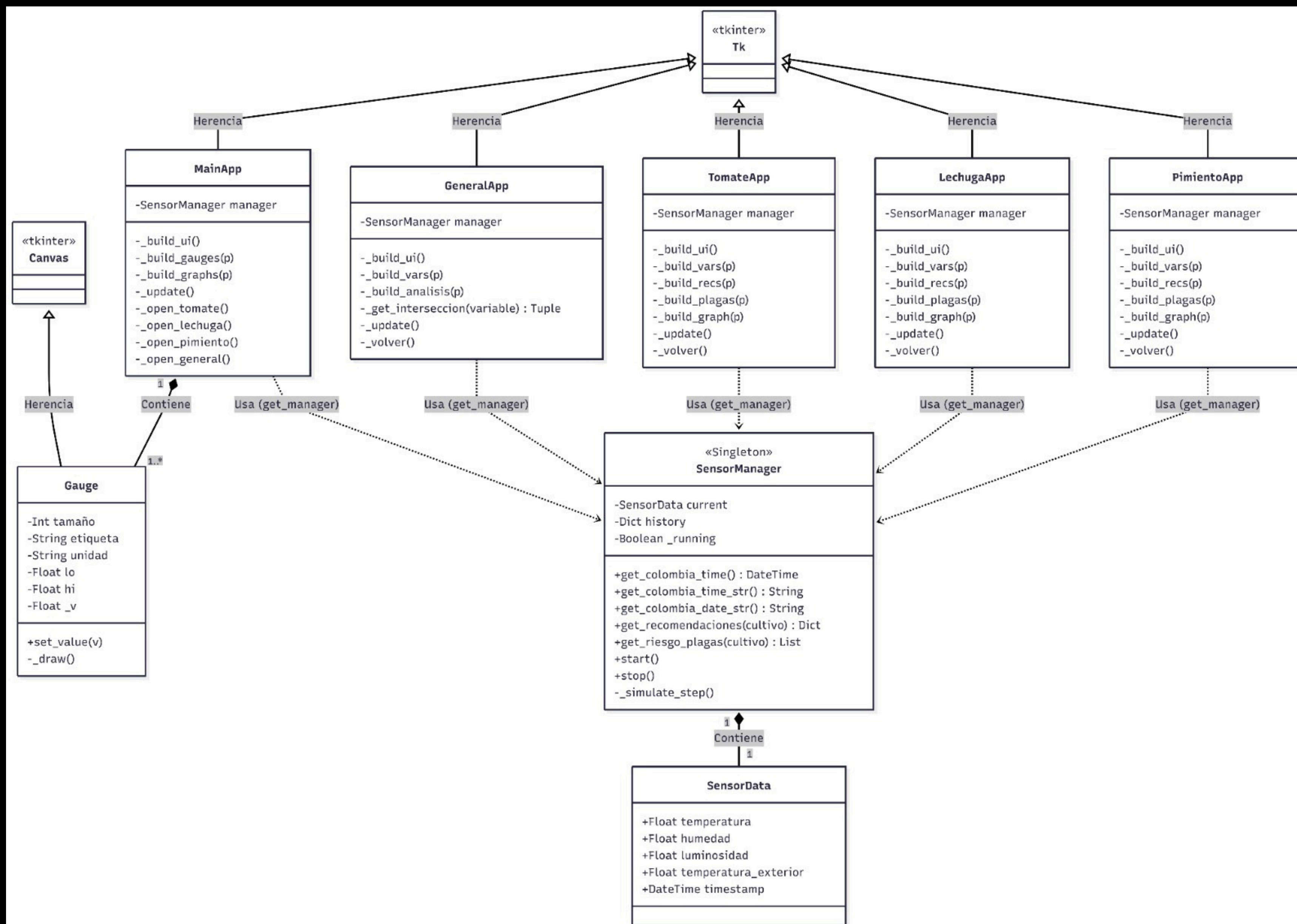
Motor Predictivo (C++)

El archivo `plagas.cpp` evalúa condiciones ambientales para calcular la probabilidad de riesgo:

- 🐛 Áfidos: Humedad > 75%
- ⚙️ Mildiu: Temp 15-20°C + Hum Alta
- 🕷️ Araña Roja: Ambiente seco y cálido

El sistema no solo monitorea, sino que **acciona** medidas preventivas basadas en el historial de datos.

Ingeniería de Software: Diagrama UML



Diseño Orientado a Objetos

La estructura jerárquica garantiza la mantenibilidad del código:

- **MainApp**: Punto de entrada y gestión de navegación.
- **SensorManager**: Singleton encargado de la persistencia y lectura de estados.
- **Specific Crop Apps**: Encapsulan la lógica particular (umbrales de temperatura y humedad).
- **SensorData**: Estructura de datos para telemetría histórica.

| Optimización del Rendimiento

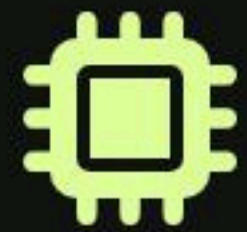
C++

Backend Engine

¿Por qué esta elección?

- **Velocidad:** Procesamiento de grandes volúmenes de telemetría en milisegundos.
- **Recursos:** Menor consumo de memoria RAM comparado con lógica pura en Python.
- **Encapsulamiento:** La lógica matemática y algorítmica queda protegida y optimizada.

| Módulos de Desarrollo



Telemetría

Implementación de `sensor_data.py` para la captura de humedad, temperatura y luminosidad.



GUI Avanzada

Uso de widgets personalizados y medidores (Gauges) para visualización inmediata.



Analítica C++

Módulos compilados en `matematica.cpp` para procesar tendencias históricas sin latencia.

| Lógica del SensorManager



Flujo de Datos

1. Lectura de `config.json` para parámetros iniciales.
2. Instanciación de hilos en `main_gui.py` para captura asíncrona.
3. Normalización de datos en la clase `SensorData`.
4. Alerta crítica si los valores exceden umbrales de seguridad.